Project Phase 4

Boilerplate Code

This document accompanies the Phase 4 problem statement. The examples use a small Mini World with three kinds of data:

- **Runners**: athletes, with a name and country;
- **Races**: events, with a name, city, date, distance and surface;
- **Results**: which runner finished which race, and their finish time in seconds. A runner can run many races, and a race has many runners.

The examples are simplified starting points to show structure and syntax. Use only the parts for the paradigm(s) you chose.

1 Schema and Population Scripts

1.1 Relational (MySQL)

schema.sql – run with `mysql -u <user> -p < schema.sql`

```
DROP DATABASE IF EXISTS running_db;
CREATE DATABASE running_db;
USE running_db;

CREATE TABLE Runners (
    runner_id INT AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    country   VARCHAR(60)  NOT NULL
);

CREATE TABLE Races (
    race_id     INT AUTO_INCREMENT PRIMARY KEY,
    race_name   VARCHAR(100) NOT NULL UNIQUE,
    city        VARCHAR(60)  NOT NULL,
    race_date   DATE         NOT NULL,
    distance_km DECIMAL(5, 2) NOT NULL CHECK (distance_km > 0),
    surface     ENUM('road', 'trail', 'track') NOT NULL
);

-- Many-to-many: one row per runner per race they finished
CREATE TABLE Results (
    runner_id       INT,
    race_id         INT,
    finish_time_sec INT NOT NULL CHECK (finish_time_sec > 0),
    PRIMARY KEY (runner_id, race_id),
    FOREIGN KEY (runner_id) REFERENCES Runners(runner_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (race_id) REFERENCES Races(race_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);
-- ... etc.
```

populate.sql – run with `mysql -u <user> -p < populate.sql`

```sql
USE running_db;

-- Parent tables first
INSERT INTO Runners (name, country) VALUES
  ('Ananya Rao',   'India'),
  ('Kiran Mehta',  'India'),
  ('Lena Fischer', 'Germany');

INSERT INTO Races (race_name, city, race_date, distance_km, surface) VALUES
  ('Lakeside 10K',          'Hyderabad', '2025-08-17', 10.00, 'road'),
  ('Hilltop Half Marathon', 'Pune',      '2025-09-21', 21.10, 'trail');

-- Then the child table that references both
INSERT INTO Results (runner_id, race_id, finish_time_sec) VALUES
  (1, 1, 2580),   -- Ananya, Lakeside 10K, 43:00
  (2, 1, 2715),   -- Kiran,  Lakeside 10K, 45:15
  (1, 2, 6420),   -- Ananya, Hilltop Half, 1:47:00
  (3, 2, 5990);   -- Lena,   Hilltop Half, 1:39:50
-- ... etc.
```

1.2 Document (MongoDB)

Here each race embeds its results. Runners are referenced, since a runner appears in many races.

schema.js – run with `mongosh schema.js`

```javascript
db = db.getSiblingDB("running_db");
db.dropDatabase();

db.createCollection("runners", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["name", "country"],
    properties: {
      name:    { bsonType: "string" },
      country: { bsonType: "string" }
    }
  }}
});

db.createCollection("races", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["race_name", "city", "race_date", "distance_km", "surface"],
    properties: {
      race_name:   { bsonType: "string" },
      city:        { bsonType: "string" },
      race_date:   { bsonType: "date" },
      distance_km: { bsonType: "number", exclusiveMinimum: 0 },
      surface:     { enum: ["road", "trail", "track"] },
      results: {
        bsonType: "array",
        items: {
          bsonType: "object",
          required: ["runner_id", "finish_time_sec"],
```

```
          properties: {
            runner_id:       { bsonType: "objectId" },
            finish_time_sec: { bsonType: "number", minimum: 1 }
          }
        }
      }
    }
  }}
});
db.races.createIndex({ race_name: 1 }, { unique: true });
db.races.createIndex({ "results.runner_id": 1 });
// ... etc.
```

populate.js – run with `mongosh populate.js`

```
db = db.getSiblingDB("running_db");

// Referenced documents first, so their _ids exist
const ananya = db.runners.insertOne({ name: "Ananya Rao",  country: "India" }).
    insertedId;
const kiran  = db.runners.insertOne({ name: "Kiran Mehta", country: "India" }).
    insertedId;

db.races.insertOne({
  race_name: "Lakeside 10K",
  city: "Hyderabad",
  race_date: new Date("2025-08-17"),
  distance_km: 10,
  surface: "road",
  results: [
    { runner_id: ananya, finish_time_sec: 2580 },
    { runner_id: kiran,  finish_time_sec: 2715 }
  ]
});
// ... etc.
```

1.3 Graph (Neo4j)

Runners and races are nodes. Each result is a `FINISHED` relationship from a runner to a race, carrying the finish time as a property.

schema.cypher – run with `cypher-shell -u neo4j -p <password> -f schema.cypher`

```
MATCH (n) DETACH DELETE n;

CREATE CONSTRAINT runner_id_unique IF NOT EXISTS
  FOR (r:Runner) REQUIRE r.runner_id IS UNIQUE;
CREATE CONSTRAINT race_name_unique IF NOT EXISTS
  FOR (r:Race) REQUIRE r.race_name IS UNIQUE;
CREATE INDEX race_date IF NOT EXISTS
  FOR (r:Race) ON (r.race_date);
// ... etc.
```

populate.cypher – run the same way

```
// Nodes first
```

```
CREATE (:Runner {runner_id: 1, name: "Ananya Rao",  country: "India"}),
       (:Runner {runner_id: 2, name: "Kiran Mehta", country: "India"});

CREATE (:Race {race_name: "Lakeside 10K", city: "Hyderabad",
               race_date: date("2025-08-17"), distance_km: 10.0,
               surface: "road"});

// Then relationships, using MATCH to find the endpoints
MATCH (a:Runner {runner_id: 1}), (race:Race {race_name: "Lakeside 10K"})
CREATE (a)-[:FINISHED {finish_time_sec: 2580}]->(race);

MATCH (k:Runner {runner_id: 2}), (race:Race {race_name: "Lakeside 10K"})
CREATE (k)-[:FINISHED {finish_time_sec: 2715}]->(race);
// ... etc.
```

2 Python Backend

The examples use Flask. FastAPI and Django (without its ORM) work the same way: a route reads the request, calls a data-access function that runs a raw query, and returns either an HTML page or JSON.

2.1 app.py (Flask + MySQL)

```
import os
import pymysql
from flask import (Flask, render_template, request, redirect,
                   url_for, flash, jsonify)

app = Flask(__name__)
app.secret_key = "change-me"


def get_db_config():
    """Database settings in one place (environment variables or defaults)."""
    return {
        "host": os.environ.get("DB_HOST", "localhost"),
        "user": os.environ.get("DB_USER", "root"),
        "password": os.environ.get("DB_PASS", ""),
        "database": os.environ.get("DB_NAME", "running_db"),
    }


def get_connection():
    return pymysql.connect(
        **get_db_config(),
        cursorclass=pymysql.cursors.DictCursor,   # rows as dictionaries
        autocommit=False,                         # commit writes manually
    )


# ---------- data access: raw, parameterised queries ----------

def search_races_mysql(q):
    """READ: races whose name contains q."""
```

```
    sql = ("SELECT race_id, race_name, city, race_date, distance_km "
           "FROM Races WHERE race_name LIKE %s ORDER BY race_date")
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(sql, (f"%{q}%",))
            return cur.fetchall()
    finally:
        conn.close()


def register_runner_mysql(name, country):
    """CREATE: add a new runner."""
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO Runners (name, country) VALUES (%s, %s)",
                (name, country),
            )
        conn.commit()
    except pymysql.Error:
        conn.rollback()
        raise
    finally:
        conn.close()


# ---------- routes: server-rendered pages ----------

@app.route("/search")
def search_page():
    q = request.args.get("q", "").strip()
    return render_template("search.html", query=q,
                           races=search_races_mysql(q))


@app.route("/runners/new", methods=["POST"])
def register_runner_page():
    name = request.form.get("name", "").strip()
    country = request.form.get("country", "").strip()
    try:
        register_runner_mysql(name, country)
        flash(f"Registered runner '{name}'.")
    except pymysql.Error as e:
        flash(f"Could not register runner: {e}")
    return redirect(url_for("search_page"))


# ---------- routes: JSON API (for a separate frontend) ----------

@app.route("/api/races")
def api_search_races():
    q = request.args.get("q", "").strip()
    return jsonify(search_races_mysql(q))

# ... add a route for every functional requirement ...
```

```python
if __name__ == "__main__":
    app.run(debug=True)
```

2.2 Frontend Option A: Server-Rendered Template

`templates/search.html` (excerpt)

```html
<form action="/search" method="get">
  <input type="text" name="q" value="{{ query }}" placeholder="Search races">
  <button type="submit">Search</button>
</form>

{% for r in races %}
  <div class="card">
    <h3>{{ r.race_name }}</h3>
    <p>{{ r.city }} &middot; {{ r.race_date }} &middot; {{ r.distance_km }} km</p>
  </div>
{% else %}
  <p>No races match "{{ query }}".</p>
{% endfor %}
```

2.3 Frontend Option B: Separate Frontend Calling the API

Any frontend (plain JavaScript, React, Vue, Svelte, ...) gets its data from the backend's JSON routes.

```javascript
async function searchRaces(q) {
  const res = await fetch(`/api/races?q=${encodeURIComponent(q)}`);
  if (!res.ok) throw new Error(`Request failed: ${res.status}`);
  return res.json();   // [{race_id, race_name, city, race_date, distance_km}, ...]
}
```

If your frontend runs on a different port from the backend (e.g. a Vite or React dev server), use the dev server's proxy setting or enable CORS on the backend (e.g. with `flask-cors`).

2.4 The Same Query in MongoDB and Neo4j

Only the data-access function changes.

PyMongo

```python
import os, re
from pymongo import MongoClient

mongo = MongoClient(os.environ.get("MONGO_URI", "mongodb://localhost:27017"))
mdb = mongo["running_db"]

def search_races_mongo(q):
    query = {"race_name": {"$regex": re.escape(q), "$options": "i"}}
    projection = {"_id": 0, "race_name": 1, "city": 1,
                  "race_date": 1, "distance_km": 1}
    return list(mdb.races.find(query, projection).sort("race_date", 1))
```

Neo4j driver

```
import os
from neo4j import GraphDatabase

driver = GraphDatabase.driver(
    os.environ.get("NEO4J_URI", "bolt://localhost:7687"),
    auth=(os.environ.get("NEO4J_USER", "neo4j"),
          os.environ.get("NEO4J_PASS", "")),
)

def search_races_neo4j(q):
    cypher = ("MATCH (r:Race) "
              "WHERE toLower(r.race_name) CONTAINS toLower($q) "
              "RETURN r.race_name AS race_name, r.city AS city, "
              "       toString(r.race_date) AS race_date, "
              "       r.distance_km AS distance_km "
              "ORDER BY race_date")
    records, _, _ = driver.execute_query(cypher, q=q, database_="neo4j")
    return [rec.data() for rec in records]
```

(`toString` turns the Neo4j date into a string, so it can be sent as JSON.)

3 Parameterised Queries

Do not build a query by inserting user input into a string. Pass user input as parameters.

3.1 Relational (PyMySQL)

Incorrect (vulnerable):

```
# DO NOT DO THIS
race_name = request.args.get("q")
cursor.execute(f"SELECT * FROM Races WHERE race_name = '{race_name}'")
```

Correct (secure):

```
# DO THIS
race_name = request.args.get("q", "").strip()
sql_query = "SELECT * FROM Races WHERE race_name = %s"
cursor.execute(sql_query, (race_name,))
```

3.2 Document (PyMongo)

Incorrect (vulnerable):

```
# DO NOT DO THIS: builds the query from a string
name = request.args.get("q")
query = json.loads('{"race_name": "' + name + '"}')
db.races.find(query)

# NOR THIS: passes user-controlled JSON straight into the filter.
# If the client sends {"name": {"$ne": null}}, every race matches.
```

```
db.races.find({"race_name": request.json["name"]})
```

Correct (secure):

```
# DO THIS: coerce input to the expected type and place it in a fixed filter
name = str(request.args.get("q", "")).strip()
db.races.find({"race_name": name})
```

Never pass user input to `$where` or other operators that execute JavaScript.

3.3 Graph (Neo4j driver)

Incorrect (vulnerable):

```
# DO NOT DO THIS
name = request.args.get("q")
session.run(f"MATCH (r:Race {{race_name: '{name}'}}) RETURN r")
```

Correct (secure):

```
# DO THIS: use $parameters
name = request.args.get("q", "").strip()
session.run("MATCH (r:Race {race_name: $name}) RETURN r", name=name)
```